

Week 04 Assignment (Node/Express - 2)

Introduction

Extend your **DevAnswers application** by implementing **User Authentication**, **Answer Management**, and **Voting** features. Build routes and services for user registration, login, posting answers, and upvote/downvote functionality.

Pre-requisites: Node.js (v24), npm, MongoDB Atlas cluster, Week 03 assignment completed

GitHub Setup

To work on this project, you'll need to fork and clone the repository to your local machine, then commit and push your changes as you work. The source code for this week's session can be found here : <https://github.com/week04-NodeExpress2>. For detailed step-by-step instructions on forking, cloning, committing, pushing, and sharing your repository, see the [GitHub Setup Guide](#)

Problem Statement

Part 1: User Service ([src/services/userService.js](#))

- **register(name, email, password, isAdmin)** - Hash passwords with bcrypt, prevent duplicate emails, return user `_id`
- **login(email, password)** - Validate credentials, generate JWT token, return `{ token, userId }`

Part 2: Authentication Middleware ([src/middleware/authHandler.js](#))

- Extract token from `Authorization` header (`Bearer <token>`)
- Verify token using `jsonwebtoken`
- Attach decoded user to `req.user`
- Return 401 if invalid/missing

Part 3: Answer Controller ([src/controllers/answerController.js](#))

- **getAnswersByQuestionId** - Retrieve all answers, populate author name
- **createAnswer** - Create answer using `req.user` as author
- **updateAnswer** - Update answer text (author/admin only)
- **deleteAnswer** - Delete answer (author/admin only)
- **upvoteAnswer** - Add user to upvotes, remove from downvotes, update voteCount
- **downvoteAnswer** - Add user to downvotes, remove from upvotes, update voteCount

Part 4: Integration

- Use `authHandler` middleware for protected routes

- Validate required fields
- Use Mongoose `populate` for author information

File Structure

```
assignment/
├── .env.example
├── .gitignore
├── vitest.config.js
├── main.js
├── server.js
├── db.js
├── package.json
├── src/
│   ├── app.js
│   ├── controllers/
│   │   ├── answerController.js
│   │   ├── authController.js
│   │   └── questionController.js
│   ├── data/
│   │   ├── answers.js
│   │   └── questions.js
│   ├── middleware/
│   │   ├── authHandler.js
│   │   └── errorHandler.js
│   ├── models/
│   │   ├── Answer.js
│   │   ├── Question.js
│   │   ├── Tag.js
│   │   └── User.js
│   ├── routes/
│   │   ├── answers.js
│   │   ├── auth.js
│   │   ├── index.js
│   │   └── questions.js
│   ├── scripts/
│   │   └── populate-db.js
│   ├── services/
│   │   ├── answerService.js
│   │   ├── questionService.js
│   │   ├── userService.js
│   │   └── voteService.js
│   └── utils/
│       └── createAppError.js
└── tests/
    ├── authController.test.js
    └── questionController.test.js
```

Functional Requirements

- **User Authentication** - Registration with password hashing, login with JWT, email uniqueness
- **Answer Management** - CRUD operations for answers, populate author info
- **Voting System** - Upvote/downvote with vote count tracking
- **Middleware & Security** - JWT authentication, token verification, protected routes

Running the Assignment

Step 1: Install Dependencies

```
npm install
```

Step 2: Add .env file with the below content inside assignment

```
PORT=3000  
NODE_ENV=development  
MONGODB_URI=<YOUR_MONGODB_URI>  
JWT_SECRET=<YOUR_SECURE_SECRET>  
JWT_EXPIRATION=7d
```

Step 3: Execute a Query

```
npm run populate # Seed database  
npm start       # Start server
```

Test endpoints:

- POST `/api/auth/register` - Register user
- POST `/api/auth/login` - Login
- GET `/api/answers/question/:questionId` - Get answers
- POST `/api/answers/question/:questionId` - Create answer (auth required)
- POST `/api/answers/:answerId/upvote` - Upvote (auth required)

Technical Topics Covered

- Express (routing, middleware), Mongoose (CRUD, populate), JWT Authentication
- Password Hashing (bcrypt), Controller-Service Pattern, Error Handling
- REST API Design, MongoDB, Async/Await, Environment Variables

Evaluation Rubric (Total: 60 Points)

- **User Service Implementation (20 points)**
- **Auth Middleware (10 points)**
- **Answer Controller (20 points)**
- **Code Organization (10 points)**

Deliverables

Submit your **GitHub repository link** containing:

- All implemented code with the required functionality
- `.env.example` file (do not commit actual `.env`)
- Proper commit history showing your development progress

Learning Outcome

Master JWT authentication, middleware concepts, MVC pattern, MongoDB relationships, voting systems, async operations, security practices (password hashing, token-based auth), RESTful API design, and production-ready configuration management.